

# OCA Java Programmer

## Complete Study Curriculum

---

|                        |                        |
|------------------------|------------------------|
| Exam Code              | 1Z0-808                |
| Full Name              | Java SE 8 Programmer I |
| Questions              | ~77 Multiple Choice    |
| Duration               | 150 Minutes            |
| Passing Score          | 65%                    |
| Recommended Study Time | 8–12 Weeks             |

# 1. Java Basics

## 1.1 — Java Structure & the JVM

- Understand the difference between JDK, JRE, and JVM
- How Java source code compiles to bytecode (.class files)
- Running a Java program with javac and java commands
- The structure of a valid Java class: package declaration, imports, class body
- Rules for file names vs class names (public class must match filename)
- The main() method signature: `public static void main(String[] args)`

## 1.2 — Data Types & Variables

| Type           | Size   | Default  | Range / Notes                       |
|----------------|--------|----------|-------------------------------------|
| <b>byte</b>    | 8-bit  | 0        | -128 to 127                         |
| <b>short</b>   | 16-bit | 0        | -32,768 to 32,767                   |
| <b>int</b>     | 32-bit | 0        | ~±2.1 billion                       |
| <b>long</b>    | 64-bit | 0L       | ~±9.2 × 10 <sup>18</sup> ; suffix L |
| <b>float</b>   | 32-bit | 0.0f     | Single precision; suffix f          |
| <b>double</b>  | 64-bit | 0.0d     | Double precision (default)          |
| <b>char</b>    | 16-bit | '\u0000' | Unicode character                   |
| <b>boolean</b> | ~1-bit | false    | true or false only                  |

- Variable naming rules: start with letter, \$, or \_; no reserved words
- Local variables must be initialized before use (no default value)
- Instance and static fields get default values automatically
- Casting: widening (automatic) vs narrowing (explicit cast required)
- Numeric literals: underscores allowed (e.g., 1\_000\_000); binary (0b), octal (0), hex (0x)

## 1.3 — Operators & Operator Precedence

- Arithmetic: +, -, \*, /, % (modulo); integer division truncates
- Unary: ++, -- (pre vs post increment — know the difference!)
- Comparison: ==, !=, <, >, <=, >= (== on objects compares references)
- Logical: && (short-circuit AND), || (short-circuit OR), ! (NOT)
- Assignment: =, +=, -=, \*=, /=, %=
- Ternary operator: `condition ? valueIfTrue : valueIfFalse`
- Bitwise: &, |, ^, ~, <<, >>, >>> (know these exist; deep mastery optional)
- Precedence order: unary → arithmetic → shift → comparison → equality → logical → ternary → assignment

## 2. Control Flow

### 2.1 — if / else Statements

- Condition must be a boolean expression (not int — unlike C/C++)
- Curly braces optional for single-statement blocks, but watch for exam tricks
- else if chaining vs nested if — understand which branch executes
- Unreachable code and missing return statements cause compiler errors

### 2.2 — switch Statements

- Supported types: byte, short, int, char, String, enum (NOT long, float, double, boolean)
- case values must be compile-time constants
- Fall-through behavior: execution continues to next case without break
- default branch can appear anywhere; executes if no case matches
- String switch is case-sensitive

### 2.3 — Loops

| Loop            | Use When                  | Key Points                                    |
|-----------------|---------------------------|-----------------------------------------------|
| <b>for</b>      | Known iteration count     | init; condition; update — any part optional   |
| <b>while</b>    | Condition checked first   | May execute 0 times                           |
| <b>do-while</b> | Run at least once         | Condition checked after body; needs semicolon |
| <b>for-each</b> | Iterating arrays/Iterable | Cannot modify index; read-only iteration      |

- break: exits the innermost loop immediately
- continue: skips the rest of the current iteration
- Labeled break/continue: can exit/skip outer loops — know the syntax
- Infinite loops: for(;;) {} or while(true) {} — must have a break or return

## 3. Arrays & Strings

### 3.1 — Arrays

- Declaration: `int[] arr;` or `int arr[];` (both valid; prefer first)
- Initialization: `new int[5]` (defaults to 0) or `{1, 2, 3}` (anonymous array)
- Arrays are objects — stored on the heap; length via `.length` (no parentheses)
- `ArrayIndexOutOfBoundsException` for invalid index access at runtime
- Multi-dimensional: `int[][] grid = new int[3][4];` — jagged arrays allowed
- `Arrays.sort()` and `Arrays.binarySearch()` — array must be sorted for `binarySearch`
- `Arrays.copyOf()` and `System.arraycopy()` for copying

### 3.2 — String

**String is immutable** — every method returns a new String; original is unchanged.

| Method                                                    | Returns              | Notes                               |
|-----------------------------------------------------------|----------------------|-------------------------------------|
| <code>length()</code>                                     | <code>int</code>     | Number of characters                |
| <code>charAt(i)</code>                                    | <code>char</code>    | Zero-based index                    |
| <code>indexOf(str)</code>                                 | <code>int</code>     | -1 if not found                     |
| <code>substring(start)</code>                             | <code>String</code>  | From start to end                   |
| <code>substring(start,end)</code>                         | <code>String</code>  | end is exclusive                    |
| <code>toLowerCase()</code> / <code>toUpperCase()</code>   | <code>String</code>  | Locale-independent versions exist   |
| <code>trim()</code>                                       | <code>String</code>  | Removes leading/trailing whitespace |
| <code>equals(str)</code>                                  | <code>boolean</code> | Case-sensitive content comparison   |
| <code>equalsIgnoreCase(str)</code>                        | <code>boolean</code> | Case-insensitive comparison         |
| <code>startsWith(str)</code> / <code>endsWith(str)</code> | <code>boolean</code> | Prefix/suffix check                 |
| <code>contains(seq)</code>                                | <code>boolean</code> | Substring check                     |
| <code>replace(old, new)</code>                            | <code>String</code>  | Replaces all occurrences            |
| <code>concat(str)</code>                                  | <code>String</code>  | Same as <code>+</code> operator     |
| <code>isEmpty()</code>                                    | <code>boolean</code> | <code>length() == 0</code>          |

### 3.3 — StringBuilder

- **StringBuilder is mutable** — modifies in place; more efficient for repeated concatenation
- Key methods: `append()`, `insert(index, str)`, `delete(start, end)`, `reverse()`, `toString()`
- `charAt()`, `indexOf()`, `length()`, `substring()` work same as `String`
- Unlike `String`, `StringBuilder` methods modify the object and return *this* (allows chaining)
- `StringBuffer` is thread-safe but slower — not heavily tested on OCA

■ **Exam Tip:** `==` on Strings compares references, not content. String literals may be pooled (same reference), but `new String("x")` always creates a new object. Always use `.equals()` for content comparison.

## 4. Object-Oriented Programming (OOP)

### 4.1 — Classes & Objects

- Class = blueprint; Object = instance created with `new`
- Instance variables (fields) vs local variables vs class variables (static fields)
- Constructors: same name as class, no return type; can be overloaded
- Default constructor: provided by compiler only when no constructor is defined
- Constructor chaining: `this()` calls another constructor in the same class; must be first statement
- `this` keyword: refers to current object instance; disambiguates field vs parameter names
- Object lifecycle: created on heap, eligible for GC when no references remain
- `finalize()` — called by GC before reclaiming memory; don't rely on it (deprecated in Java 9)

### 4.2 — Encapsulation & Access Modifiers

| Modifier         | Same Class | Same Package | Subclass | Everywhere |
|------------------|------------|--------------|----------|------------|
| <b>private</b>   | ✓          | ✗            | ✗        | ✗          |
| <b>(default)</b> | ✓          | ✓            | ✗        | ✗          |
| <b>protected</b> | ✓          | ✓            | ✓        | ✗          |
| <b>public</b>    | ✓          | ✓            | ✓        | ✓          |

- Encapsulation = private fields + public getters/setters
- JavaBeans convention: `getX()`, `setX()`, `isX()` for booleans

### 4.3 — Inheritance & Polymorphism

- Inheritance via `extends`; Java supports single inheritance for classes
- `super`: refers to parent class; `super()` calls parent constructor (must be first statement)
- Method overriding: same signature, covariant return type allowed, can't reduce visibility
- `@Override` annotation — use it; compiler catches signature mismatches
- Polymorphism: reference type determines which methods are *visible*; object type determines which override *runs*
- Casting: upcasting (implicit, always safe); downcasting (explicit, may throw `ClassCastException`)
- `instanceof`: check type before casting to avoid `ClassCastException`
- `final` class cannot be extended; `final` method cannot be overridden
- Abstract class: declared with `abstract`; can't be instantiated; may have abstract methods
- Abstract method: no body; subclass must implement it (unless subclass is also abstract)

### 4.4 — Interfaces

- Declared with `interface`; implemented with `implements`
- A class can implement multiple interfaces (solves the multiple-inheritance problem)
- Interface methods are implicitly `public abstract`; fields are `public static final`

- Java 8+: interfaces can have default methods (with body) and static methods
- Functional interface: exactly one abstract method — used with lambda expressions
- If two implemented interfaces have the same default method, the class must override it

## 5. Exception Handling

### 5.1 — Exception Hierarchy

Throwable → Error (don't catch) | Exception → RuntimeException (unchecked) | Other Exceptions (checked)

| Category            | Examples                                                                                                             | Must Handle?                  |
|---------------------|----------------------------------------------------------------------------------------------------------------------|-------------------------------|
| Checked Exceptions  | IOException, SQLException, FileNotFoundException                                                                     | Yes — catch or declare throws |
| Unchecked (Runtime) | NullPointerException, ArrayIndexOutOfBoundsException, ClassCastException, NumberFormatException, ArithmeticException | No — optional to handle       |
| Errors              | StackOverflowError, OutOfMemoryError                                                                                 | No — don't catch these        |

### 5.2 — try / catch / finally

- try block: code that might throw an exception
- catch block: handles specific exception type; multiple catch blocks allowed (most specific first)
- Multi-catch: `catch (IOException | SQLException e)` — pipe-separated types
- finally block: always executes (except `System.exit()`); used for cleanup (close resources)
- If both catch and finally throw exceptions, the finally exception wins
- try-with-resources (Java 7+): `try (Resource r = new Resource()) {}` — auto-closes resources
- Resources must implement `AutoCloseable`; closed in reverse order of declaration

### 5.3 — Throwing & Declaring Exceptions

- throw: throws an exception instance — `throw new IllegalArgumentException("msg");`
- throws: declares checked exceptions a method may throw — part of the method signature
- Checked exceptions must be caught or declared; forgetting causes compile error
- You can throw a subclass of a declared exception (Liskov for exceptions)
- Rethrowing: catch an exception, do something, then throw it again

■ *Exam Tip: catch blocks must go from most specific to most general. Catching a parent before a child is a compile error.*

## 6. Core Java API

### 6.1 — ArrayList & the List Interface

- Import: `java.util.ArrayList` and `java.util.List`
- Declare with interface type: `List<String> list = new ArrayList<>();` (diamond operator)
- `add(element)`, `add(index, element)`, `set(index, element)`, `remove(index)`, `remove(Object)`
- `get(index)`, `size()`, `isEmpty()`, `contains(Object)`, `indexOf(Object)`
- `Collections.sort(list)` for sorting; `Collections.reverse()`, `Collections.shuffle()`
- Autoboxing: primitives auto-convert to wrapper types (`int` → `Integer`) and back
- Cannot use primitives as type parameters — use wrapper classes (`Integer`, `Double`, etc.)

### 6.2 — Wrapper Classes & Autoboxing

| Primitive | Wrapper   | Parse Method                           | toString / valueOf                |
|-----------|-----------|----------------------------------------|-----------------------------------|
| int       | Integer   | <code>Integer.parseInt(str)</code>     | <code>Integer.toString(n)</code>  |
| long      | Long      | <code>Long.parseLong(str)</code>       | <code>Long.valueOf(str)</code>    |
| double    | Double    | <code>Double.parseDouble(str)</code>   | <code>Double.toString(d)</code>   |
| boolean   | Boolean   | <code>Boolean.parseBoolean(str)</code> | <code>Boolean.valueOf(str)</code> |
| char      | Character | N/A                                    | <code>Character.isDigit(c)</code> |

### 6.3 — Lambda Expressions (Java 8)

- Syntax: `(parameters) -> expression` or `(parameters) -> { statements; }`
- Used where a functional interface (single abstract method) is expected
- Common functional interfaces: `Predicate<T>`, `Consumer<T>`, `Supplier<T>`, `Function<T,R>`
- Example: `Predicate<String> p = s -> s.length() > 5;`
- Can be used with `removeIf()`, `forEach()`, `sort()` on collections
- Lambda can't modify non-effectively-final local variables from enclosing scope



## 7. Exam Strategy & Resources

### 7.1 — Study Timeline (8–12 Weeks)

| Week(s) | Focus Area                                 | Goal                                            |
|---------|--------------------------------------------|-------------------------------------------------|
| 1–2     | Java Basics, Variables, Operators          | Comfortable with all primitives and expressions |
| 3–4     | Arrays, Strings, StringBuilder             | Know all key methods; understand immutability   |
| 5–6     | OOP: Classes, Inheritance, Interfaces      | Understand polymorphism and casting deeply      |
| 7–8     | Exceptions & Java API (ArrayList, Lambdas) | Handle all exception scenarios correctly        |
| 9–10    | Full Mock Exams                            | Score 75%+ consistently before the real exam    |
| 11–12   | Review Weak Areas & Final Prep             | Focus on tricky topics; rest before exam day    |

### 7.2 — Recommended Resources

| Resource                                                     | Type           | Why It's Great                                                 |
|--------------------------------------------------------------|----------------|----------------------------------------------------------------|
| OCA Java SE 8 Programmer I Study Guide (Boyarsky & Selikoff) | Book           | The gold standard; written by exam experts; covers every topic |
| Enthuware Mock Exams                                         | Practice Tests | Closest to real exam style; detailed explanations              |
| Oracle's Official Exam Page                                  | Website        | Free sample questions; official topic list                     |
| Udemy OCA Courses                                            | Video          | Good for visual learners; pick highly-rated ones               |
| CodeRanch Forums (javaranch.com)                             | Community      | Exam-specific Q&A; active Java community                       |

### 7.3 — Top Exam Tips

- ✓ Read every question twice — one keyword (static, final, private) changes the answer
- ✓ Compile mentally first — ask 'does this even compile?' before guessing the output
- ✓ Know == vs .equals() cold — this appears in multiple questions
- ✓ Understand variable scope — where variables are declared determines where they live
- ✓ Know constructor chaining rules — this() and super() placement matters
- ✓ Practice output-tracing questions — write code, predict output, then run it
- ✓ Time yourself — aim for ~2 minutes per question maximum
- ✓ Flag and return — skip hard questions, answer all easier ones first
- ✓ Score 75%+ on Enthuware before scheduling your real exam

---

**Good luck on your OCA exam! Consistent daily practice is the key to success.**